```
Finished in 0.108150s, 18.4929 runs/s, 258.9002 assertions/s…
2 runs, 28 assertions, 0 failures, 0 errors, 0 skips
```

The reason we had to call `refresh` in our tests has to do with how the UI works. Right after the `CompleteOrderJob` is queued, the controller returns and shows the order's show view. You may have noticed that as the third-party services are called, various attributes like `email_id` or `charge_decline_reason` are set on the order. The view knows that these values may be blank.

In order to properly assert the view's behavior *after* the Sidekiq job has executed, you have to refresh the page. You could make the page auto-refresh, but for now, you can manually refresh it just to see everything working.

This does demonstrate a reason why Fake mode is preferable: before refreshing the page, you could make assertions about how the order's show view should appear while the job is running. Once you've completed the assertions, call `drain_all`, then `refresh` to proceed.

Before we leave this section, a test of the `OrderCreator` is now failing. We can use the same technique we just did to get these tests to pass. First, add `Sidekiq::Worker.drain_all` and a `reload` to the test for a decline:

snapshots/1-5/sidekiq-book/test/services/order_creator_test.rb

```
    )
    resulting_order = @order_creator.create_order(order)
    assert_equal order, resulting_order, "should return the same order"
»   Sidekiq::Job.drain_all
»   resulting_order.reload
    refute resulting_order.charge_successful
    assert_equal "Insufficient funds", resulting_order.charge_decline_reason
    assert_nil resulting_order.charge_id
```

Now, make a similar change in the test for success:

snapshots/1-6/sidekiq-book/test/services/order_creator_test.rb

```
    resulting_order = @order_creator.create_order(order)
    assert_equal order, resulting_order, "should return the same order"
```